

CITS5508 Practice Exam — Chapters 1-2 — ANSWER KEY

Question 1.

(a) (3 marks) For a film recommender: - **T** (task) = recommend/predict films the user will enjoy (e.g. predict a rating or the next film watched). - **E** (experience) = the training data — the user's (and other users') past viewing/ratings history. - **P** (performance measure) = e.g. recommendation accuracy / RMSE of predicted ratings / click-through or watch rate on recommendations. (1 mark each; any sensible, consistent triple.)

(b) (4 marks) - **Supervised**: training data includes the **labels/targets**; the model learns a mapping from features to target. **Unsupervised**: data is **unlabelled**; the model finds structure on its own. - *Supervised example*: email spam classification — **features** = word-count/bag-of-words vector; **response** = spam/ham (a class label). - *Unsupervised example*: customer segmentation by clustering — **features** = purchase history/activity; **no response variable** (clusters are discovered). (2 marks for the distinction, 1 mark per correctly described example.)

(c) (3 marks, 1 each) 1. **Reinforcement learning** — an agent learns a policy from rewards/penalties by trial. 2. **Unsupervised learning** — clustering, no predefined categories/labels. 3. **Self-supervised learning** — labels (the masked pixels) are generated from the data itself, then used like supervised targets.

(d) (2 marks) - **Instance-based**: “learns by heart” — stores training examples and generalises using a **similarity measure** at prediction time (e.g. **k-NN**). - **Model-based**: builds a **model with parameters** from the data and predicts with it (e.g. **linear regression**). (1 mark for distinction, 1 mark for two valid examples.)

Question 2.

(a) (3 marks) A **parameter** is **learned from the data during training** (e.g. the weights θ of a linear model) and defines the trained model. A **hyperparameter** is a setting of the **learning algorithm, fixed before training and not changed by training** (e.g. regularisation strength α , k in k -NN, tree `max_depth`). Key distinction: parameters are learned; hyperparameters are set in advance.

(b) (5 marks) - **Overfitting** = the model performs well on training data but **generalises poorly** (too complex relative to the data/noise; fits noise). Remedies (any two): simplify/constrain the model (fewer parameters/features, more regularisation); gather more training data; reduce noise (clean data, remove outliers). - **Underfitting** = the model is **too simple** to capture the structure, so it's inaccurate even on training data. Remedies (any two): use a more powerful model (more parameters); better features (feature engineering); reduce regularisation/constraints. (1 mark each definition; 1 mark for one remedy and $\frac{1}{2} + \frac{1}{2}$ — i.e. ~ 1.5 marks of remedies per type; award full marks for a clear definition + two valid remedies each.)

(c) (4 marks) **Regularisation** = constraining a model to make it simpler and reduce overfitting. It reduces the model's **degrees of freedom** — the number of ways it can adapt to the data (e.g. forcing weights to stay small lies between full freedom and zero freedom). A **very large** regularisation hyperparameter over-constrains the model (e.g. weights ≈ 0 , an almost-flat model), so it can no longer fit the real structure → **high bias / underfitting**.

Question 3.

(a) (4 marks) If you tune/select using the **test set** repeatedly, you end up choosing whatever happens to be best **for that specific set** — effectively fitting to the test set — so the reported error is **optimistically biased** and the model underperforms in production. A **validation set** (held out from the **training** data) is used to compare candidates; the **test set is touched only once** at the very end to give an unbiased generalisation estimate.

(b) (4 marks) **k-fold CV**: split the training set into k folds; train k times, each time holding out a different fold for evaluation; average the k scores. - **Advantage**: a **more reliable/precise** performance estimate (every instance is used for both training and validation; you also get a variance estimate). - **Disadvantage**: **$k \times$ more training time** (computationally expensive).

(c) (4 marks) Hold out some of the **web (training-distribution)** images as a **train-dev set**. Train on the training set only (not train-dev), then: - Poor on **train-dev** → the model **overfit** the training data (simplify/regularise, get more/cleaner data). - Good on train-dev but poor on the **dev (phone-photo) set** → the issue is **data mismatch** (pre-process web images to look like phone photos, then retrain). This separates overfitting from mismatch because train-dev and training come from the *same* distribution, while dev/test come from the production distribution.

Question 4.

(a) (5 marks) Errors ($\hat{y} - y$): +2, -2, +5, -1, -12.

- **MAE** = $(|2| + |-2| + |5| + |-1| + |-12|) / 5 = (2 + 2 + 5 + 1 + 12) / 5 = 22 / 5 = 4.4$.
- **Squared errors**: 4, 4, 25, 1, 144 → sum = 178. **MSE** = $178 / 5 = 35.6$. **RMSE** = $\sqrt{35.6} \approx 5.97$. (2 marks MAE, 3 marks RMSE incl. MSE step.)

(b) (3 marks) **RMSE** is more sensitive to outliers, because squaring inflates large errors (the -12 dominates). RMSE corresponds to the ℓ_2 (**Euclidean**) **norm**; MAE corresponds to the ℓ_1 (**Manhattan**) **norm**. (Note $RMSE\ 5.97 \gg MAE\ 4.4$ here precisely because of the outlier.)

(c) (2 marks) **Data-snooping bias** = looking at / analysing the test data influences your modelling choices, so your brain “overfits” to it and the generalisation estimate becomes over-optimistic. Prevention: **set the test set aside first**, before any exploration, and use it only once at the end.

(d) (2 marks) **Stratified sampling** splits the data so the train and test sets preserve the **proportions of important strata** (subgroups) present in the full dataset. It beats random sampling when the dataset isn’t huge and a feature is important, because random sampling can by chance over-/under-represent a subgroup, biasing the test estimate.

Question 5.

(a) (4 marks) `fit_transform` on the training set **learns** the scaling statistics (mean μ and std σ) **from the training data** and applies them. `transform` on the test set applies those **same** learned statistics. If you `fit_transform` the test set too, you learn **new statistics from the test data** → information about the test set **leaks** into pre-processing (**data leakage**), so the evaluation no longer reflects true generalisation (over-optimistic), and train/test would be scaled inconsistently.

(b) (4 marks) - Numeric feature with missing values → **impute**, e.g. `SimpleImputer`.

puter(strategy="median") (fit the median on the training set only). - Nominal categorical feature → **OneHotEncoder** (one binary feature per category). - An **OrdinalEncoder** maps categories to integers 0,1,2..., which implies an **ordering / numeric distance** between categories; the model would wrongly assume nearby integers are “similar” categories, which is meaningless for nominal data.

(c) (4 marks)

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

pipe = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LinearRegression()
)
pipe.fit(X_train, y_train)
```

Benefit (any one): the whole sequence is applied **identically and automatically** to training, test and new/live data (no leakage, no manual re-application); fewer mistakes; the pipeline can be cross-validated/grid-searched as a single estimator. (3 marks for correct pipeline, 1 mark for a valid benefit.)